

OSGi Working Group
OSGi Specification repo for
`org.osgi.service.jndiRelease`

1.0.0-SNAPSHOT

September 2026



Copyright © 2000, 2026 Eclipse Foundation

LICENSE

Eclipse Foundation Specification License – v1.0

By using and/or copying this document, or the Eclipse Foundation document from which this statement is linked, you (the licensee) agree that you have read, understood, and will comply with the following terms and conditions:

Permission to copy, and distribute the contents of this document, or the Eclipse Foundation document from which this statement is linked, in any medium for any purpose and without fee or royalty is hereby granted, provided that you include the following on ALL copies of the document, or portions thereof, that you use:

- link or URL to the original Eclipse Foundation document.
- All existing copyright notices, or if one does not exist, a notice (hypertext is preferred, but a textual representation is permitted) of the form: "Copyright © [date-of-document] Eclipse Foundation, Inc. <<url to this license>>".

Inclusion of the full text of this NOTICE must be provided. We request that authorship attribution be provided in any software, documents, or other items or products that you create pursuant to the implementation of the contents of this document, or any portion thereof.

No right to create modifications or derivatives of Eclipse Foundation documents is granted pursuant to this license, except anyone may prepare and distribute derivative works and portions of this document in software that implements the specification, in supporting materials accompanying such software, and in documentation of such software, PROVIDED that all such works include the notice below. HOWEVER, the publication of derivative works of this document for use as a technical specification is expressly prohibited.

The notice is:

"Copyright © [date-of-document] Eclipse Foundation. This software or document includes material copied from or derived from [title and URI of the Eclipse Foundation specification document]."

Disclaimers

THIS DOCUMENT IS PROVIDED "AS IS," AND THE COPYRIGHT HOLDERS AND THE ECLIPSE FOUNDATION MAKE NO REPRESENTATIONS OR WARRANTIES, EXPRESS OR IMPLIED, INCLUDING, BUT NOT LIMITED TO, WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE, NON-INFRINGEMENT, OR TITLE; THAT THE CONTENTS OF THE DOCUMENT ARE SUITABLE FOR ANY PURPOSE; NOR THAT THE IMPLEMENTATION OF SUCH CONTENTS WILL NOT INFRINGE ANY THIRD PARTY PATENTS, COPYRIGHTS, TRADEMARKS OR OTHER RIGHTS.

THE COPYRIGHT HOLDERS AND THE ECLIPSE FOUNDATION WILL NOT BE LIABLE FOR ANY DIRECT, INDIRECT, SPECIAL OR CONSEQUENTIAL DAMAGES ARISING OUT OF ANY USE OF THE DOCUMENT OR THE PERFORMANCE OR IMPLEMENTATION OF THE CONTENTS THEREOF.

The name and trademarks of the copyright holders or the Eclipse Foundation may NOT be used in advertising or publicity pertaining to this document or its contents without specific, written prior permission. Title to copyright in this document will at all times remain with copyright holders.

Preface

Implementation Requirements

An implementation of a Specification: (i) must fully implement the Specification including all its required interfaces and functionality; (ii) must not modify, subset, superset or otherwise extend the OSGi Name Space, or include any public or protected packages, classes, Java interfaces, fields or methods within the OSGi Name Space other than those required and authorized by the Specification. An implementation that does not satisfy limitations (i) (ii) is not considered an implementation of the Specification and must not be described as an implementation of the Specification. "OSGi Name Space" shall mean the public class or interface declarations whose names begin with "org.osgi" or any recognized successors or replacements thereof. An implementation of a Specification must not claim to be a compatible implementation of the Specification unless it passes the Technology Compatibility Kit ("TCK") for the Specification.

Feedback

This specification can be downloaded from the OSGi Documentation web site:

<https://docs.osgi.org/specification/>

Comments about this specification can be raised at:

<https://github.com/osgi/osgi/issues>

Table of Contents

1	Introduction	5
1.1	Reader Level	5
1.2	Version Information	5
1.3	References	5
126	JNDI Services Specification	7
126.1	Introduction	7
126.2	JNDI Overview	10
126.3	JNDI Context Manager Service	12
126.4	JNDI Provider Admin service	14
126.5	JNDI Providers	15
126.6	OSGi URL Scheme	18
126.7	Traditional Client Model	20
126.8	Security	22
126.9	org.osgi.service.jndi	23
126.10	References	25

Licensed under the Eclipse Foundation Specification License – v1.0. Copyright © Contributors to the Eclipse Foundation

DRAFT

1 Introduction

This document contains a specification developed by the OSGi working group.

1.1 Reader Level

This specification is written for the following audiences:

- Application developers
- Framework and system service developers (system developers)
- Architects

This specification assumes that the reader has at least one year of practical experience in writing Java programs. Experience with embedded systems and server-environments is a plus. Application developers must be aware that the OSGi environment is significantly more dynamic than traditional desktop or server environments.

System developers require a very deep understanding of Java. At least three years of Java coding experience in a system environment is recommended. A Framework implementation will use areas of Java that are not normally encountered in traditional applications. Detailed understanding is required of class loaders, garbage collection, Java 2 security, and Java native library loading.

Architects should focus on the introduction of each subject. This introduction contains a general overview of the subject, the requirements that influenced its design, and a short description of its operation as well as the entities that are used. The introductory sections require knowledge of Java concepts like classes and interfaces, but should not require coding experience.

Most of these specifications are equally applicable to application developers and system developers.

1.2 Version Information

Each OSGi specification has an independent version, and packages defined by those specifications have their own versions as well. Specification and package versions follow semantic versioning rules.

1.3 References

- [1] *OSGi Specifications*
<https://docs.osgi.org/specification/>

DRAFT

Licensed under the Eclipse Foundation Specification License – v1.0. Copyright © Contributors to the Eclipse Foundation

126 JNDI Services Specification

Version 1.0

126.1 Introduction

Naming and directory services have long been useful tools in the building of software systems. The ability to use a programming interface to publish and consume objects can provide many benefits to any system. The Java Naming and Directory Interface (JNDI) is a registry technology in Java applications, both in the Java SE and Java EE space. JNDI provides a vendor-neutral set of APIs that allow clients to interact with a naming service from different vendors.

The JNDI as used in the Java SE environment relies on the class loading model provided by the JDK to find providers. By default, it attempts to load the JNDI provider class using the Thread Context Class Loader. In an OSGi environment, this type of Context creation is not desirable since it relies on the JNDI provider classes being visible to the JNDI client, or require it to set the Context Class Loader; in both cases breaking modularity. For modularity reasons, it is important that clients are not required to express a dependency on the implementation of services they use.

This specification will define how JNDI can be utilized from within an OSGi framework. The specification consists of three key parts:

- *OSGi Service Model* - How clients interact with JNDI when running inside an OSGi Framework.
- *JNDI Provider Model* - How JNDI providers can advertise their existence so they are available to OSGi and traditional clients.
- *Traditional Model* - How traditional JNDI applications and providers can continue to work in an OSGi Framework without needing to be rewritten when certain precautions are taken.

126.1.1 Essentials

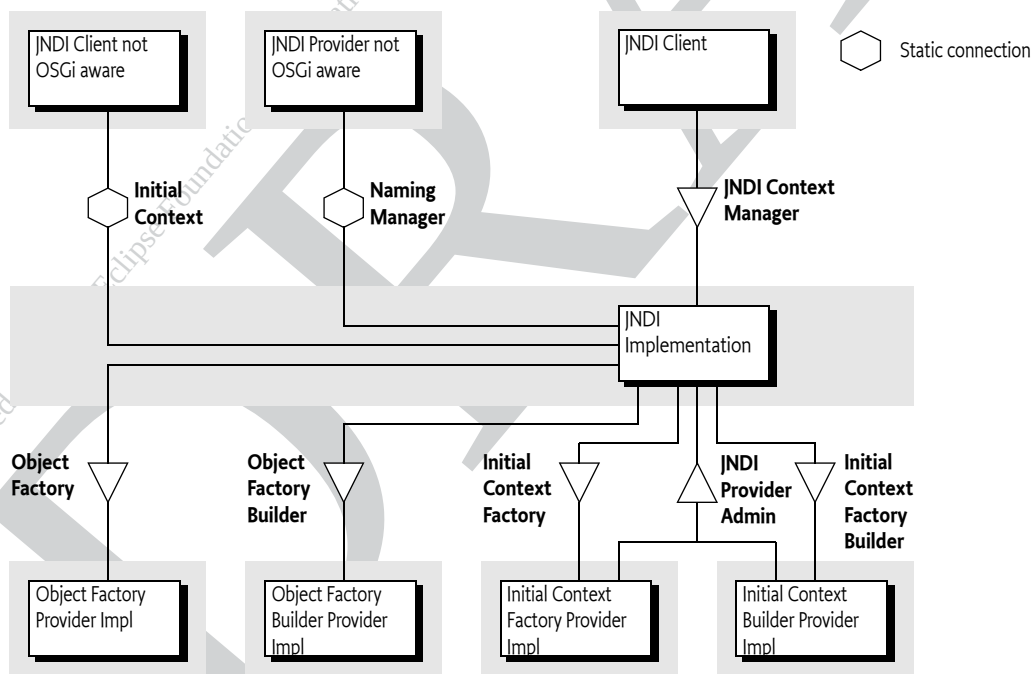
- *Naming Service* - Provide an integration model for JNDI API clients and providers.
- *Flexible* - Provide a standard mechanism for publishing and locating JNDI providers.
- *Compatibility* - Support the traditional JNDI programming model used by Java SE and Java EE clients.
- *Service Based* - Provide a service model that clients and providers can use to leverage JNDI facilities.
- *Migration* - Provide a mechanism to access OSGi services from a JNDI context.

126.1.2 Entities

- *JNDI Implementation* - The Implementer of the JNDI Context Manager, JNDI Provider Admin, and setter of the JNDI static singletons.
- *JNDI Client* - Any code running within an OSGi bundle that needs to use JNDI.
- *JNDI Context Manager* - A service that allows clients to obtain Contexts via a service.
- *JNDI Provider Admin* - A service that allows the conversion of objects for providers.
- *JNDI Provider* - Provides a Context implementation.
- *Context* - A Context abstracts a namespace. Implementations are provided by JNDI providers and the Contexts are used by JNDI clients. The corresponding interface is `javax.naming.Context`.

- *Dir Context* - A sub-type of Context that provides mechanisms for examining and updating the attributes of an object in a directory structure, and for performing searches in an hierarchical naming systems like LDAP. The corresponding interface is `javax.naming.directory.DirContext`.
- *Initial Context Factory* - A factory for creating instances of Context objects. This factory is used to integrate new JNDI Providers. In general, a single Initial Context Factory constructs Context objects for a single provider implementation. The corresponding interface is `javax.naming.spi.InitialContextFactory`.
- *Initial Context Factory Builder* - A factory for InitialContextFactory objects. A single Initial Context Factory Builder can construct InitialContextFactory objects for different types of Contexts. The interface is `javax.naming.spi.InitialContextFactoryBuilder`.
- *Object Factory* - Used in conversion of objects. The corresponding interface is `javax.naming.spi.ObjectFactory`.
- *Dir Object Factory* - An Object Factory that takes attribute information for object conversion. The corresponding interface is `javax.naming.spi.DirObjectFactory`.
- *Object Factory Builder* - A factory for ObjectFactory objects. A single Object Factory Builder can construct ObjectFactory instances for different types of conversions. The corresponding interface is `javax.naming.spi.ObjectFactoryBuilder`.
- *Reference* - A description of an object that can be turned into an object through an Object Factory. The associated Referenceable interface implemented on an object indicates that it can provide a Reference object.

Figure 126.1 JNDI Service Specification Service Entities



126.1.3 Dependencies

The classes and interfaces used in this specification come from the following packages:

```

javax.naming
javax.naming.spi
javax.naming.directory

```

These packages have no associated version. It is assumed they come from the runtime environment. This specification is based on Java SE 1.4 or later.

126.1.4 Synopsis

A client bundle wishing to make use of JNDI in order to access JNDI Providers such as LDAP or DNS in OSGi should not use the Naming Manager but instead use the JNDI Context Manager service. This service can be asked for a Context based on environment properties. The environment properties are based on an optional argument in the `newInitialContext` method, the Java System properties, and an optional resource in the caller's bundle.

These environment properties can specify an implementation class name for a factory that can create a Context object. If such a class name is specified, then it is searched for in the service registry. If such a service is found, then that service is used to create a new Context, which is subsequently returned. If no class name is specified, the service registry is searched for Initial Context Factory services. These services are tried in ranking order, as specified in `ServiceReference.compareTo`, to see if they can create an appropriate Context, the first one that can create a Context is then used.

If no class name is specified, all Initial Context Factory Builder services are tried to see if they can create a Context, the first non-null result is used. If no Context can be found, a `NoInitialContextException` is thrown. Otherwise, the JNDI Context Manager service returns an initial Context that uses the just created Context from a provider as the backing service. This initial Context delegates all operations to this backing Context, except operations that use a name that can be interpreted as a URL, that is, the name contains a colon. URL operations are delegated a URL Context that is associated with the used scheme. URL Contexts are found through the general object conversion facility provided by the JNDI Provider Admin service.

The JNDI Provider Admin service provides a general object conversion facility that can be extended with Object Factory and Object Factory Builder services that are traditionally provided through the Naming Manager `getObjectInstance` method. A specific case for this conversion is the use of Reference objects. Reference objects can be used to store objects persistently in a Context implementation. Reference objects must be converted to their corresponding object when retrieved from a Context.

During the client's use of a Context it is possible that its provider's service is unregistered. In this case the JNDI Context Manager must release the backing Context. If the initial Context is used and no backing Context is available, the JNDI Context Manager must re-create a new Context, if possible. Otherwise a `NamingException` is thrown. If subsequently a proper new backing Context can be created, the initial Context must start operating again.

The JNDI Context Manager service must track the life cycle of a calling bundle and ensure that any returned Context objects are closed and returned objects are properly cleaned up when the bundle is closed or the JNDI Context Manager service is unget.

When the client bundle is stopped, any returned initial Context objects are closed and discarded. If the Initial Context Factory, or Initial Context Factory Builder, service that created the initial Context goes away then the JNDI Context Manager service releases the Context backing the initial Context and attempts to create a replacement Context.

Clients and JNDI Context providers that are unaware of OSGi use static methods to connect to the JRE JNDI implementation. The `InitialContext` class provides access to a Context from a provider and providers use the static `NamingManager` methods to do object conversion and find URL Contexts. This traditional model is not aware of OSGi and can therefore only be used reliably if the consequences of this lack of OSGi awareness are managed.

126.2 JNDI Overview

The Java Naming and Directory Interface (JNDI) provides an abstraction for namespaces that is included in Java SE. This section describes the basic concepts of JNDI as provided in Java SE. These concepts are later used in the service model provided by this specification.

126.2.1 Context and Dir Context

The [1] *Java Naming and Directory Interface* (JNDI) defines an API for *namespaces*. These namespaces are abstracted with the Context interface. Namespaces that support *attributes*, such as a namespace as the Lightweight Directory Access Protocol (LDAP), are represented by the DirContext class, which extends the Context class. If applicable, a Context object can be cast to a DirContext object. The distinction is not relevant for this specification, except in places where it is especially mentioned.

The Context interface models a set of name-to-object *bindings* within a namespace. These bindings can be looked-up, created, and updated through the Context interface. The Context interface can be used for federated, flat, or hierarchical namespaces.

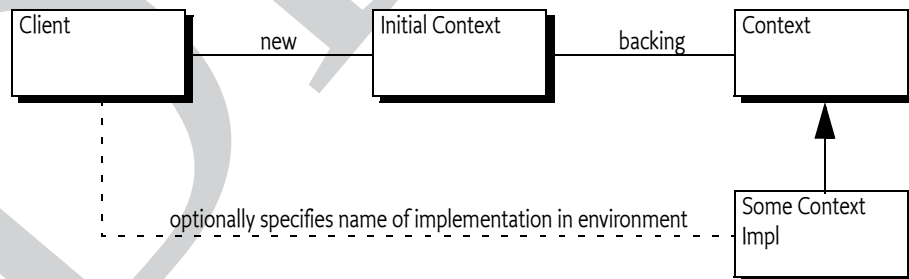
126.2.2 Initial Context

Obtaining a Context for a specific namespace, for example DNS, is handled through the InitialContext class. Creating an instance of this class will cause the JRE to find a *backing* Context. The Initial Context is only a facade for the backing Context. The facade context provides URL based lookups.

The backing Context is created by a *JNDI Provider*. How this backing Context is created is an elaborate process using class loading techniques or a provisioning mechanism involving *builders*, see *Naming Manager Singletons* on page 11 for more information about the builder provisioning mechanism.

If there is no Initial Context Factory Builder set, the class name of a class implementing the InitialContextFactory interface is specified as a property in the *environment*. The environment is a Hashtable object that is constructed from different sources and then merged with System properties and a resource in the calling bundle, see *Environment* on page 11. In a standard Java SE JNDI, the given class name is then used to construct an InitialContextFactory object and this object is then used to create the backing Context. This process is depicted in Figure 126.2 on page 10.

Figure 126.2 Backing Context



126.2.3 URL Context Factory

The InitialContext class implements the Context interface. It can therefore delegate all the Context interface methods to the backing Context object. However, it provides a special URL lookup behavior for names that are formed like URLs, that is, names that contain a colon (':') character. This behavior is called a *URL lookup*.

URL lookups are not delegated to the backing Context but are instead first tried via a *URL Context* based lookup on the given scheme, like:

`myscheme: foo`

For example a lookup using `acme:foo/javax.sql.DataSource` results in a URL Context being used, rather than the backing Context.

JNDI uses class loading techniques to search for an `ObjectFactory` class that can be used to create this URL Context. The Naming Manager provides a static method `getURLContext` for this purpose. If such a URL Context is found, it is used with the requested operation and uses the full URL. If no such URL Context can be found, the backing Context is asked to perform the operation with the given name.

The URL lookup behavior is only done when the backing Context was created by the JNDI implementation in the JRE. If the backing Context had been created through the singleton provisioning mechanism, then no URL lookup is done for names that have a colon. The URL lookup responsibility is then left to the backing Context implementation.

126.2.4 Object and Reference Conversion

The `NamingManager` class provides a way to create objects from a *description* with the `getObjectInstance` method. In general, it will iterate over a number of `ObjectFactory` objects and ask each one of them to provide the requested object. The first non-null result indicates success. These `ObjectFactory` objects are created from an environment property.

A special case for the description argument in the `getObjectInstance` method is the *Reference*. A Reference is a description of an object that can be stored persistently. It can be re-created into an actual object through the static `getObjectInstance` method of the `NamingManager` class. The Reference object describes the actual `ObjectFactory` implementing class that must be used to create the object.

This default behavior is completely replaced with the Object Factory Builder singleton by getting the to be used `ObjectFactory` object directly from the set singleton Object Factory Builder.

126.2.5 Environment

JNDI clients need a way to set the configuration properties to select the proper JNDI Provider. For example, a JNDI Provider might require an identity and a password in order to access the service. This type of configuration is referred to as the *environment* of a Context. The environment is a set of properties. Common property names can be found in [3] *JNDI Standard Property Names*. The set of properties is build from the following sources (in priority order, that is later entries are shadowed by earlier entries):

1. Properties set in the environment Hashtable object given in the constructor argument (if any) of the `InitialContext` class.
2. Properties from the Java System Properties
3. Properties found in `$JAVA_HOME/lib/jndi.properties`

There are some special rules around the handling of specific properties.

126.2.6 Naming Manager Singletons

The default behavior of the JRE implementation of JNDI can be extended in a standardized way. The `NamingManager` class has two static singletons that allow JNDI Providers outside the JRE to provide `InitialContextFactory` and `ObjectFactory` objects. These singletons are set with the following static methods on the `NamingManager` class:

- `setObjectBuilderFactoryBuilder(ObjectFactoryBuilder)` - A hook to provide `ObjectFactory` objects.
- `setInitialContextFactoryBuilder(InitialContextFactoryBuilder)` - A hook to provide `InitialContextFactory` objects. This hook is consulted to create a Context object that will be associated with an `InitialContext` object the client creates.

These JNDI Provider hooks are *singletons* and must be set *before* any application code creates an `InitialContext` object or any objects are converted. If these singletons are not set, the JNDI implementation in the JRE will provide a default behavior that is based on searching through classes defined in an environment property.

Both singletons can only be set once. A second attempt to set these singletons results in an `IllegalStateException` being thrown.

126.2.7 Built-In JNDI Providers

The Java Runtime Environment (JRE) defines the following default providers:

- *LDAP* - Lightweight Directory Access Protocol (LDAP) service provider
- *COS* - CORBA Object Service (COS) naming service provider
- *RMI* - Remote Method Invocation (RMI) Registry service provider
- *DNS* - Domain Name System (DNS) service provider

Although these are the default JNDI Service Providers, the JNDI architecture provides a number of mechanisms to plug-in new types of providers.

126.3 JNDI Context Manager Service

The JNDI Context Manager service allows clients to obtain a Context using the OSGi service model. By obtaining a JNDI Context Manager service, a client can get a Context object so that it can interact with the available JNDI Providers. This service replaces the approach where the creation of a new `InitialContext` object provided the client with access to an `InitialContext` object that was backed by a JNDI Provider's Context.

The `JNDIContextManager` interface defines the following methods for obtaining Context objects:

- `newInitialContext()` - Obtain a Context object using the default environment properties.
- `newInitialContext(Map)` - Get a Context object using the default environment properties merged with the given properties.
- `newInitialDirContext()` - Get a `DirContext` object using a default environment properties.
- `newInitialDirContext(Map)` - Get a `DirContext` object using the default environment properties merged with the given properties.

The JNDI Context Manager service returns Context objects that implement the same behavior as the `InitialContext` class; the returned Context object does not actually extend the `InitialContext` class, its only guarantee is that it implements the Context interface.

This Context object is a facade for the context that is created by the JNDI Provider. This JNDI Provider's Context is called the *backing Context*. This is similar to the behavior of the `InitialContext` class. However, in this specification, the facade can change or loose the backing Context due to the dynamics of the OSGi framework.

The returned facade must also provides URL lookups, just like an Initial Context. However, the URL Context lookup must be based on Object Factory services with a service property that defines the scheme.

The environment properties used to create the backing Context are constructed in a similar way as the environment properties of the Java SE JNDI, see *Environment and Bundles* on page 13 .

The following sections define in detail how a JNDI Provider Context must be created and managed.

126.3.1 Environment and Bundles

The Java SE JNDI looks for a file in `$JAVAHOME/lib/jndi.properties`, see *Environment* on page 11. A JNDI Implementation must not use this information but it must use a resource in the bundle that uses the JNDI Context Manager service. The order is therefore:

1. Properties set in the environment Hashtable object given in the constructor argument (if any) of the InitialContext class.
2. Properties from the Java System Properties
3. A properties resource from the bundle that uses the service called `/jndi.properties`.

The following four properties do not overwrite other properties but are merged:

- `java.naming.factory.object`
- `java.naming.factory.state`
- `java.naming.factory.control`
- `java.naming.factory.url.pkgs`

These property values are considered lists and the ultimate value used by the JNDI Providers is taken by merging the values found in each stage into a single colon separated list. For more information see [3] *JNDI Standard Property Names*.

The environment consists of the merged properties. This environment is then passed to the Initial Context Factory Builder for the creation of an Initial Context Factory.

126.3.2 Context Creation

When a client calls one of the `newInitialContext` (or `newInitialDirContext`) methods, the JNDI Context Manager service must construct an object that implements the Context interface based on the environment properties. All factory methods in the `InitialContextFactory` and `InitialContextFactoryBuilder` classes take a Hashtable object with the environment as an argument, see *Environment and Bundles* on page 13.

The caller normally provides a specific property in the environment that specifies the class name of a provider class. This property is named:

```
java.naming.factory.initial
```

The algorithm to find the provider of the requested Context can differ depending on the presence or absence of the `java.naming.factory.initial` property in the environment.

In the following sections the cases for presence or absence of the `java.naming.factory.initial` property are described. Several steps in these algorithm iterate over a set of available services. This iteration must always take place in ranking order as specified in `ServiceReference.compareTo`.

Exception handling in the following steps is as follows:

- If an Exception is thrown by an Initial Context Factory Builder service, then this Exception must be logged but further ignored.
- Exceptions thrown by the InitialContextFactory objects when creating a Context must be thrown to the caller.

126.3.2.1 Implementation Class Present in Environment

If the implementation class is specified, a JNDI Provider is searched in the service registry with the following steps, which stop when a backing Context can be created:

1. Find a service in ranking order that has a name matching the given implementation class name as well as the InitialContextFactory class name. The searching must take place through the Bundle Context of the requesting bundle but must not require that the requesting bundle imports

the package of the implementation class. If such a matching Initial Context Factory service is found, it must be used to construct the Context object that will act as the backing Context.

2. Get all the Initial Context Factory Builder services. For each such service, in ranking order:
 - Ask the Initial Context Factory Builder service to create a new InitialContextFactory object. If this is null then continue with the next service.
 - Create the Context with the found Initial Context Factory and return it.
3. If no backing Context could be found using these steps, then the JNDI Context Manager service must throw a No Initial Context Exception.

126.3.2.2

No Implementation Class Specified

If the environment does not contain a value for the `java.naming.factory.initial` property then the following steps must be used to find a backing Context object.

1. Get all the Initial Context Factory Builder services. For each such service, in ranking order, do:
 - Ask the Initial Context Factory Builder service to create a new InitialContextFactory object. If this is null, then continue with the next service.
 - Create the backing Context object with the found Initial Context Factory service and return it.
2. Get all the Initial Context Factory services. For each such service, in ranking order, do:
 - Ask the Initial Context Factory service to create a new Context object. If this is null then continue with the next service otherwise create a new Context with the created Context as the backing Context.
3. If no Context has been found, an initial Context is returned without any backing. This returned initial Context can then only be used to perform URL based lookups.

126.3.3

Rebinding

A JNDI Provider can be added or removed to the service registry at any time because it is an OSGi service; OSGi services are by their nature dynamic. When a JNDI Provider unregisters an Initial Context Factory that was used to create a backing service then the JNDI Context Manager service must remove the association between any returned Contexts and their now invalid backing Contexts.

The JNDI Context Manager service must try to find a replacement whenever it is accessed and no backing Context is available. However, if no such replacement can be found the called function must result in throwing a No Initial Context Exception.

126.3.4

Life Cycle and Dynamism

When a client has finished with a Context object, then the client must close this Context object by calling the close method. When a Context object is closed, the resources held by the JNDI Implementation on the client's behalf for that Context must all be released. Releasing these resources must not affect other, independent, Context objects returned to the same client.

If a client ungets the JNDI Context Manager service, all the Context objects returned through that service instance must automatically be closed by the JNDI Context Manager. When the JNDI Context Manager service is unregistered, the JNDI Context Manager must automatically close all Contexts held.

For more information about life cycle issues, see also *Life Cycle Mismatch* on page 21 .

126.4 JNDI Provider Admin service

JNDI provides a general object conversion service, see *Object and Reference Conversion* on page 11 . For this specification, the responsibility of the static method on the NamingManager getObjectIn-

stance is replaced with the JNDI Provider Admin service. The `JNDIProviderAdmin` interface provides the following methods that can be used to convert a description object to an object:

- `getObjectInstance(Object,Name,Context,Map)` - Used by Context implementations to convert a description object to another object.
- `getObjectInstance(Object,Name,Context,Map,Attributes)` - Used by a Dir Context implementations to convert a description object to another object.

In either case, the first argument is an object, called the *description*. JNDI allows a number of different Java types here. When either method is called, the following algorithm is followed to find a matching Object Factory to find/create the requested object. This algorithm is identical for both methods, except that the call that takes the `Attributes` argument consults Dir Object Factory services first and then Object Factory services while the method without the `Attributes` parameter only consults Object Factory services.

1. If the description object is an instance of `Referenceable`, then get the corresponding `Reference` object and use this as the description object.
2. If the description object is not a `Reference` object then goto step 5.
3. If a factory class name is specified, the JNDI Provider Admin service uses its own `Bundle Context` to search for a service registered under the `Reference`'s factory class name. If a matching Object Factory is found then it is used to create the object from the `Reference` object and the algorithm stops here.
4. If no factory class name is specified, iterate over all the `Reference` object's `StringRefAddr`s objects with the address type of `URL`. For each matching address type, use the value to find a matching `URL Context`, see *URL Context Provider* on page 17, and use it to recreate the object. See the *Naming Manager* for details. If an object is created then it is returned and the algorithm stops here.
5. Iterate over the Object Factory Builder services in ranking order. Attempt to use each such service to create an `ObjectFactory` or `DirObjectFactory` instance. If this succeeds (non null) then use this `ObjectFactory` or `DirObjectFactory` instance to recreate the object. If successful, the algorithm stops here.
6. If the description was a `Reference` and without a factory class name specified, or if the description was not of type `Reference`, then attempt to convert the object with each Object Factory service (or Dir Object Factory service for directories) service in ranking order until a non-null value is returned.
7. If no `ObjectFactory` implementations can be located to resolve the given description object, the description object is returned.

If an Exception occurs during the use of an Object Factory Builder service then this exception should be logged but must be ignored. If, however, an Exception occurs during the calling of a found `ObjectFactory` or `DirObjectFactory` object then this Exception must be re-thrown to the caller of the JNDI Provider Admin service.

126.5 JNDI Providers

JNDI Providers can be registered by registering an appropriate service. These services are consulted by the JNDI Implementation for creating a Context as well as creating/finding/converting general objects.

126.5.1 Initial Context Factory Builder Provider

An Initial Context Factory Builder provider is asked to provide an Initial Context Factory when no implementation class is specified or no such implementation can be found. An Initial Context Fac-

tory Builder service can be used by containers for other bundles to control the initial Context their applications receive.

An Initial Context Factory Builder provider must register an Initial Context Factory Builder service. The iteration ordering of multiple Initial Context Factory Builder services must always take place in ranking order as specified in `ServiceReference.compareTo`. Implementations must be careful to correctly provide defaults.

For example, a container could use a thread local variable to mark the stack for a specific application. The implementation of the Initial Context Factory Builder can then detect specific calls from this application. To make the next code example work, an instance must be registered as an Initial Context Factory Builder service.

```
public class Container implements InitialContextFactoryBuilder {
    ThreadLocal<Application> apps;

    void startApp(final Application app) {
        Thread appThread = new Thread(app.getName()) {
            public void run() {
                apps.set(app);
                app.run();
            }
        };
    }

    public InitialContextFactory
    createInitialContextFactory( Hashtable<?,?> ht ) {
        final Application app = apps.get();
        if ( app == null )
            return null;

        return new InitialContextFactory() {
            public Context getInitialContext( Hashtable<?,?>env ) {
                return app.getContext(env);
            }
        };
    }
}
```

126.5.2 Initial Context Factory Provider

An Initial Context Factory provides Contexts of a specific type. For example, those contexts allow communications with an LDAP server. An Initial Context Factory Provider must register the its Initial Context Factory service under the following names:

- *Implementation Class* - An Initial Context Factory provider must register a service under the name of the implementation class. This allows the JNDI Context Manager to find implementations specified in the environment properties.
- *Initial Context Factory* - As a general Initial Context Factory. If registered as such, it can be consulted for a default Initial Context. Implementations must be careful to only return a Context when the environment properties are appropriate. See *No Implementation Class Specified* on page 14

An Initial Context Factory service can create both `DirContext` as well as `Context` objects.

For example, SUN JREs for Java SE provide an implementation of a Context that can answer DNS questions. The name of the implementation class is a well known constant. The following class can be used with Declarative Services to provide a lazy implementation of a DNS Context:

```
public class DNSProvider implements InitialContextFactory{
    public Context createInitialContextFactory( Hashtable<?,?>env ) throws
```

```

        NamingException {
        try {
            Class<InitialContextFactory> cf = (Class<InitialContextFactory>)
                1.loadClass("com.sun.jndi.dns.DnsContextFactory" );
            InitialContextFactory icf = cf.newInstance();
            return icf.createInitialContextFactory(env);
        } catch( Throwable t ) {
            return null;
        }
    }
}

```

126.5.3 Object Factory Builder Provider

An Object Factory Builder provider must register an Object Factory Builder service. Such a service can be used to provide ObjectFactory and/or DirObjectFactory objects. An Object Factory Builder service is requested for such an object when no specific converter can be found. This service can be leveraged by bundles that act as a container for other bundles to control the object conversion for their subjects.

126.5.4 Object Factory Provider

An Object Factory provider can participate in the conversion of objects. It must register a service under the following names:

- *Implementation Class* - A service registered under its implementation class can be leveraged by a description that is a Reference object. Such an object can contain the name of the factory class. The implementation class can implement the DirObjectFactory interface or the ObjectFactory interface.
- *Object Factory* - The ObjectFactory interface is necessary to ensure class space consistency.
- *Dir Object Factory* - If the Object Factory provider can accept the additional Attributes argument in the getObjectInstance method of the JNDI Provider Admin service than it must also register as a Dir Object Factory service.

126.5.5 URL Context Provider

A *URL Context Factory* is a special type of an Object Factory service. A URL Context Factory must be registered as an Object Factory service with the following service property:

- *osgi.jndi.url.scheme* - The URL scheme associated with this URL Context, for example acme. The scheme must not contain the colon (':') (\u003A).

A URL Context is used for URL based operations on an initial Context. For example, a lookup to acme:foo/javax.sql.DataSource must not use the provider based lookup mechanism of the backing Context but instead causes a lookup for the requested URL Context. A URL Context also provides a secondary mechanism for restoring Reference objects.

When an initial Context returned by the JNDI Context Manager service is given a URL based operation, it searches in the service registry for an Object Factory service that is published with the URL scheme property that matches the scheme used from the lookup request.

It then calls the getInstance method on the Object Factory service with the following parameters:

- *Object* - Should be either a String, String[], or null.
- *Name* - must be null
- *Context* - must be null
- *Hashtable* - The environment properties.

Calling the `getInstance` method must return a Context object. This context is then used to perform the lookup.

The life cycle of the Object Factory used to create the URL Context is tied to the JNDI context that was used to perform the URL based JNDI operation. By the time JNDI context is closed any Object-Factory objects held to process the URL lookups must be released (`unset`).

126.5.6 JRE Context Providers

The Java Runtime Environment (JRE) defines a number of default naming providers., see *Built-In JNDI Providers* on page 12 . These naming providers are not OSGi aware, but are commonly used and are provided by the JRE. These naming providers rely on the `NamingManager` class for object conversion and finding URL Contexts.

The JRE default providers are made available by the JNDI Implementation. This JNDI Implementation must register a *built-in* Initial Context Factory Builder service that is capable of loading any `InitialContextFactory` classes of the JRE providers.

When this built-in Initial Context Factory Builder is called to create an `InitialContextFactory` object it must look in the environment properties that were given as an argument and extract the `java.naming.factory.initial` property; this property contains the name of the class of a provider. The built-in Initial Context Factory Builder then must use the bootstrap class loader to load the given `InitialContextFactory` class and creates a new instance with the no arguments constructor and return it. If this fails, it must return null. This mechanism will allow loading of any built-in providers.

This built-in Initial Context Factory Builder service must be registered with no `service.ranking` property. This will give it the default ranking and allows other providers to override the default.

126.6 OSGi URL Scheme

A URL scheme is available that allows JNDI based applications to access services in the service registry, see *Services and State* on page 20 about restrictions on these services. The URL scheme is specified as follows:

```
service ::= 'osgi:service/' query
query   ::= jndi-name | qname ( '/' filter )?
jndi-name ::= <any string>
```

No spaces are allowed between the terms.

This OSGi URL scheme can be used to perform a lookup of a single matching service using the interface name and filter. The URL Context must use the *owning bundle* to perform the service queries. The owning bundle is the bundle that requested the initial Context from the JNDI Context Manager service or received its Context through the `InitialContext` class. The returned objects must not be incompatible with the class space of the owning bundle.

The lookup for a URL with the `osgi:` scheme and service path returns the service whose `ServiceReference` is first in ranking order as specified in `ServiceReference.compareTo`. This scheme only allows a single service to be found. Multiple services can be obtained with the `osgi:` scheme and `servicelist` path:

```
servicelist ::= 'osgi:servicelist/' query?
```

If this `osgi:servicelist` scheme is used from a lookup method then a Context object is returned instead of a service object. Calling the `listBindings` method will produce a `NamingEnumeration` object that provides `Binding` objects. A `Binding` object contains the name, class of the service, and the service object. The bound object is the service object contained in the given Context.

When the Context class list method is called, the Naming Enumeration object provides a NameClassPair object. This NameClassPair object will include the name and class of each service in the Context. The list method can be useful in cases where a client wishes to iterate over the available services without actually getting them. If the service itself is required, then listBindings method should be used.

If multiple services matched the criteria listed in the URL, there would be more than one service available in the Context, and the corresponding Naming Enumeration would contain the same number of services.

If multiple services match, a call to listBindings on this Context would return a list of bindings whose name are a string with the service.id number, for example:

1283

Thus the following lookup is valid:

```
osgi:servicelist/javax.sql.DataSource/(&(db=mydb)(version=3.1))
```

A service can provide a *JNDI service name* if it provides the following service property:

- `osgi.jndi.service.name` - An alternative name that the service can be looked up by when the `osgi:` URL scheme is used.

If a service is published with a JNDI service name then the service matches any URL that has this service name in the place of interface. For example, if the JNDI service name is `foo`, then the following URL selects this service:

```
osgi:service/foo
```

Using a JNDI service name that can be interpreted as an interface name must be avoided, if this happens the result is undefined.

A JNDI client can also obtain the Bundle Context of the owning bundle by using the `osgi:scheme namespace` with the `framework/bundleContext` name. The following URL must return the Bundle Context of the owning bundle:

```
osgi:framework/bundleContext
```

After the NamingEnumeration object has been used it must be closed by the client. Implementations must then unget any gotten services or perform other cleanup.

126.6.1

Service Proxies

The OSGi URL Context handles the complexities by hiding the dynamic nature of OSGi. The OSGi URL Context must handle the dynamics by *proxying* the service objects. This proxy must implement the interface given in the URL. If the JNDI service name instead of a class name is used, then all interfaces under which the service is registered must be implemented. If an interface is not compatible with the owning bundle's class space then it must not be implemented on the proxy, it must then be ignored. If this results in no implemented interfaces then an Illegal Argument Exception must be thrown.

Interfaces can always be proxied but classes are much harder. For this reason, an implementation is free to throw an Illegal Argument Exception when a class is used in the URL or in one of the registration names.

Getting the actual service object can be delayed until the proxy is actually used to call a method. If a method is called and the actual service has been unregistered, then the OSGi URL Context must attempt to rebind it to another service that matches the criteria given in the URL the next time it is

called. When no alternative service is available, a Service Exception with the UNREGISTERED type code must be thrown. Services obtained with the osgi: URL scheme must therefore be stateless because the rebinding to alternative services is not visible to the caller; there are no listeners defined for this rebinding, see *Services and State* on page 20 .

If the reference was looked up using osgi:service!list then proxies must still be used, however, these proxies must not rebind when their underlying service is unregistered. Instead, they must throw a Service Exception with the UNREGISTERED type whenever the proxy is used and the proxied service is no longer available.

126.6.2 Services and State

A service obtained through a URL Context lookup is proxied. During the usage of this service, the JNDI Implementation can be forced to transparently rebind this service to another instance. The JNDI specification is largely intended for portability. For this reason, it has no mechanism architected to receive notifications about this rebinding. The client code is therefore unable to handle the dynamics.

The consequence of this model is that stateful services require extra care because applications cannot rely on the fact that they always communicate with the same service. Virtually all OSGi specified services have state.

126.7 Traditional Client Model

A JNDI Implementation must at startup register the InitialContextFactoryBuilder object and the ObjectFactoryBuilder object with the NamingManager class. As described in *JNDI Overview* on page 10 , the JNDI code in the JRE will then delegate all Context related requests to the JNDI Implementation. Setting these singletons allows code that is not aware of the OSGi framework to use Context implementations from JNDI Providers registered with the OSGi service registry and that are managed as bundles. The JNDI Implementation therefore acts as a broker to the service registry for OSGi unaware code.

This brokering role can only be played when the JNDI Implementation can set the singletons as specified in *Naming Manager Singletons* on page 11 . If the JNDI Implementation cannot set these singletons then it should log an error with the Log Service, if available. It can then not perform the following sections.

126.7.1 New Initial Context

The client typically requests a Context using the following code:

```
Hashtable env = new Hashtable();
env.put(Context.INITIAL_CONTEXT_FACTORY, "com.sun.jndi.ldap.LdapCtxFactory");
InitialContext ctx = new InitialContext(env);
```

The created InitialContext object is a facade for the real Context that is requested by the caller. It provides the bootstrapping mechanism for JNDI Provider plugability. In order to obtain the provider's Context, the InitialContext class makes a call to the static getContext method on the NamingManager class. The JNDI code in the JRE then delegates any request for an initial Context object to the JNDI Implementation through the registered InitialContextFactoryBuilder singleton. The JNDI Implementation then determines the Bundle Context of the caller as described in *Caller's Bundle Context* on page 21 . If no such Bundle Context can be found, a No Initial Context Exception is thrown to the caller. This Bundle Context must be from an ACTIVE bundle.

This Bundle Context is then used to get the JNDI Context Manager service. This service is then used as described in *Context Creation* on page 13 to get an initial Context. This initial Context is then used in the InitialContext object as the *default initial context*. In this specification this is normal-

ly called the backing context. An InitialContext object constructed through an Initial Context Factory Builder will not use the URL lookup mechanism, it must delegate all operations to the its backing context. A Context obtained through the JNDI Context Manager provides the URL lookup behavior instead.

126.7.2 Static Conversion

JNDI provides a general object conversion facility that is used by the URL Context and the process of restoring an object from a Reference object, see *Object and Reference Conversion* on page 11 . A JNDI Implementation must take over this conversion by setting the static Object Factory Builder singleton, see *Naming Manager Singletons* on page 11 . Non-OSGi aware Context implementations will use the NamingManager static getObjectInstance method for object conversion. This method then delegates to the set singleton Object Factory Builder to obtain an ObjectFactory object that understands how to convert the given description to an object. The JNDI Implementation must return an Object Factory that understands the OSGi service registry. If the getObjectInstance method is called on this object it must use the same rules as defined for the JNDI Provider Admin service `getObjectInstance(Object, javax.naming.Name, javax.naming.Context, Map)` method, see *JNDI Provider Admin service* on page 14 . The Bundle Context that must be used with respect to this service is the caller's Bundle Context, see *Caller's Bundle Context* on page 21 . If the Bundle Context is not found, the description object must be returned. The calling bundle must not be required to import the `org.osgi.service.jndi` package.

126.7.3 Caller's Bundle Context

The following mechanisms are used to determine the callers Bundle Context:

1. Look in the JNDI environment properties for a property called `org.osgi.service.jndi.bundleContext`

If a value for this property exists then use it as the Bundle Context. If the Bundle Context has been found stop.
2. Obtain the Thread Context Class Loader; if it, or an ancestor class loader, implements the BundleReference interface, call its `getBundle` method to get the client's Bundle; then call `getBundleContext` on the Bundle object to get the client's Bundle Context. If the Bundle Context has been found stop.
3. Walk the call stack until the invoker is found. The invoker can be the caller of the InitialContext class constructor or the NamingManager or DirectoryManager getObjectInstance methods.
 - Get the class loader of the caller and see if it, or an ancestor, implements the BundleReference interface.
 - If a Class Loader implementing the BundleReference interface is found call the `getBundle` method to get the clients Bundle; then call the `getBundleContext` method on the Bundle to get the clients Bundle Context.
 - If the Bundle Context has been found stop, else continue with the next stack frame.

126.7.4 Life Cycle Mismatch

The use of static access to the JNDI mechanisms, NamingManager and InitialContext class methods, in the traditional client programming model produces several problems with regard to the OSGi life cycle. The primary problem being that there is no dependency management in place when static methods are used. These problems do not exist for the JNDI Context Manager service. Therefore, OSGi applications are strongly encouraged to use the JNDI Context Manager service.

The traditional programming model approach relies on two JVM singletons in the Naming Manager, see *Naming Manager Singletons* on page 11 . The JNDI Implementation bundle must set both singletons before it registers its JNDI Context Manager service and JNDI Provider Admin service. However, in OSGi there is no defined start ordering, primarily because bundles can be updated

at any moment in time and will at such time not be available to provide their function anyway. For this reason, OSGi bundles express their dependencies with services.

The lack of start ordering means that a bundle could create an `InitialContext` object before the JNDI Implementation has had the chance to set the static Initial Context Factory Builder singleton. This means that the JNDI implementation inside the JRE will provide its default behavior and likely have to throw an exception. A similar exception is thrown for the Object Factory Builder singleton.

There is also a (small) possibility that a client will call `new InitialContext()` after the singletons have been set, but before the JNDI Context Manager and JNDI Provider Admin services have been registered. This specification requires that these services are set after the singletons are set. In this race condition the JNDI Implementation should throw a `No Initial Context Exception`, explaining that the JNDI services are not available yet.

126.8 Security

126.8.1 JNDI Implementation

A JNDI Implementation may wish to assert that the user of the provider has some relevant Java 2 security permission. Since the JNDI implementation is an intermediary between the JNDI client and provider this means that the JNDI implementation needs to have any permissions required to access any JNDI Provider. As a result the JNDI implementation needs `All Permission`. This will result in the JNDI clients permissions being checked to see if it has the relevant permission to access the JNDI Provider.

The JNDI Implementation must make any invocation to access these services in a `doPrivileged` check. A JNDI client must therefore not be required to have the following permissions, which are needed by a JNDI Implementation:

<code>ServicePermission ..ObjectFactory</code>	<code>REGISTER, GET</code>
<code>ServicePermission ..DirObjectFactory</code>	<code>REGISTER, GET</code>
<code>ServicePermission ..ObjectFactoryBuilder</code>	<code>REGISTER, GET</code>
<code>ServicePermission ..InitialContextFactory</code>	<code>REGISTER, GET</code>
<code>ServicePermission ..InitialContextFactoryBuilder</code>	<code>REGISTER, GET</code>
<code>ServicePermission ..JNDIProviderAdmin</code>	<code>REGISTER, GET</code>

The JNDI Implementation bundle must have the appropriate permissions to install the `InitialContextFactoryBuilder` and `ObjectFactoryBuilder` instances using the appropriate methods on the `NamingManager` class. This requires the following permission:

```
RuntimePermission "setFactory"
```

126.8.2 JNDI Clients

A JNDI client using the JNDI Context Manager service must have the following permissions:

<code>ServicePermission ..JNDIContextManager</code>	<code>GET</code>
---	------------------

Obtaining a reference to a JNDI Context Manager service should be considered a privileged operation and should be guarded by permissions.

126.8.3 OSGi URL namespace

A JNDI client must not be able to obtain services or a Bundle Context that the client bundle would not be able to get via the core OSGi API. To allow a client to use the `osgi` namespace to get a service the bundle must have the corresponding Service Permission. When using the `osgi` namespace to obtain the Bundle Context the client bundle must have `Admin Permission` for the Bundle Context.

These permissions must be enforced by the osgi URL namespace handler. If there is no proper permission, the implementation must throw a Name Not Found Exception to prevent exposing the existence of such services.

126.9 org.osgi.service.jndi

JNDI Package Version 1.0.

Bundles wishing to use this package must list the package in the Import-Package header of the bundle's manifest. This package has two types of users: the consumers that use the API in this package and the providers that implement the API in this package.

Example import for consumers using the API in this package:

```
{@code Import-Package: org.osgi.service.jndi; version="[1.0,2.0)"}
```

Example import for providers implementing the API in this package:

```
{@code Import-Package: org.osgi.service.jndi; version="[1.0,1.1)"}
```

126.9.1 Summary

- **JNDIConstants** - Constants for the JNDI implementation.
- **JNDIContextManager** - This interface defines the OSGi service interface for the JNDIContextManager.
- **JNDIProviderAdmin** - This interface defines the OSGi service interface for the JNDIProviderAdmin service.

126.9.2 public class JNDIConstants

Constants for the JNDI implementation.

Concurrency Immutable

126.9.2.1 public static final String BUNDLE_CONTEXT = "org.osgi.service.jndi.bundleContext"

This JNDI environment property can be used by a JNDI client to indicate the caller's BundleContext. This property can be set and passed to an InitialContext constructor. This property is only useful in the "traditional" mode of JNDI.

126.9.2.2 public static final String JNDI_SERVICENAME = "org.osgi.jndi.service.name"

This service property is set on an OSGi service to provide a name that can be used to locate the service other than the service interface name.

126.9.2.3 public static final String JNDI_URLSCHEME = "org.osgi.jndi.url.scheme"

This service property is set by JNDI Providers that publish URL Context Factories as OSGi Services. The value of this property should be the URL scheme that is supported by the published service.

126.9.3 public abstract JNDIContextManager

This interface defines the OSGi service interface for the JNDIContextManager. This service provides the ability to create new JNDI Context instances without relying on the InitialContext constructor.

Concurrency Thread-safe

126.9.3.1 public abstract Context newInitialContext() throws NamingException

- Creates a new JNDI initial context with the default JNDI environment properties.

Returns an instance of `javax.naming.Context`

Throws `NamingException`—upon any error that occurs during context creation

126.9.3.2 **public abstract Context newInitialContext(Map<String, ?> environment) throws NamingException**

environment JNDI environment properties specified by caller

- Creates a new JNDI initial context with the specified JNDI environment properties.

Returns an instance of `javax.naming.Context`

Throws `NamingException`—upon any error that occurs during context creation

126.9.3.3 **public abstract DirContext newInitialDirContext() throws NamingException**

- Creates a new initial `DirContext` with the default JNDI environment properties.

Returns an instance of `javax.naming.directory.DirContext`

Throws `NamingException`—upon any error that occurs during context creation

126.9.3.4 **public abstract DirContext newInitialDirContext(Map<String, ?> environment) throws NamingException**

environment JNDI environment properties specified by the caller

- Creates a new initial `DirContext` with the specified JNDI environment properties.

Returns an instance of `javax.naming.directory.DirContext`

Throws `NamingException`—upon any error that occurs during context creation

126.9.4 **public abstract JNDIProviderAdmin**

This interface defines the OSGi service interface for the `JNDIProviderAdmin` service. This service provides the ability to resolve JNDI References in a dynamic fashion that does not require calls to `{@code NamingManager.getObjectInstance()}`. The methods of this service provide similar reference resolution, but rely on the OSGi Service Registry in order to find `{@code ObjectFactory}` instances that can convert a Reference to an Object. This service will typically be used by OSGi-aware JNDI Service Providers.

Concurrency Thread-safe

126.9.4.1 **public abstract Object getObjectInstance(Object refInfo, Name name, Context context, Map<String, ?> environment) throws Exception**

refInfo Reference info

name the JNDI name associated with this reference

context the JNDI context associated with this reference

environment the JNDI environment associated with this JNDI context

- Resolve the object from the given reference.

Returns an Object based on the reference passed in, or the original reference object if the reference could not be resolved.

Throws `Exception`—in the event that an error occurs while attempting to resolve the JNDI reference.

126.9.4.2 **public abstract Object getObjectInstance(Object refInfo, Name name, Context context, Map<String, ?> environment, Attributes attributes) throws Exception**

refInfo Reference info

name the JNDI name associated with this reference

context the JNDI context associated with this reference

- environment* the JNDI environment associated with this JNDI context
- attributes* the naming attributes to use when resolving this object
- Resolve the object from the given reference.
- Returns* an Object based on the reference passed in, or the original reference object if the reference could not be resolved.
- Throws* Exception—in the event that an error occurs while attempting to resolve the JNDI reference.

126.10 References

- [1] *Java Naming and Directory Interface*
<https://docs.oracle.com/javase/8/docs/technotes/guides/jndi/index.html>
- [2] *Java Naming and Directory Interface Tutorial*
<https://docs.oracle.com/javase/tutorial/jndi/index.html>
- [3] *JNDI Standard Property Names*
<https://docs.oracle.com/javase/8/docs/api/javax/naming/Context.html>

Licensed under the Eclipse Foundation Specification License – v1.0. Copyright © Contributors to the Eclipse Foundation

DRAFT

End Of Document

Licensed under the Eclipse Foundation Specification License – v1.0. Copyright © Contributors to the Eclipse Foundation

DRAFT